

WEEK 9 PRACTICAL

QUESTION 1

Implement a file copy utility using low-level Linux file I/O system calls (open, read, write, lseek, close). Compare its performance with standard library functions (fopen, fread, fwrite).

Aim

To implement a file copy utility in C using the low-level system calls **open()**, **read()**, **write()**, **lseek()** and **close()**, and to compare its performance with the buffered standard I/O library functions **fopen()**, **fread()** and **fwrite()**.

Program

```
10  /*
11  #include <stdio.h>
12  #include <stdlib.h>
13  #include <string.h>
14  #include <errno.h>
15  #include <fcntl.h>
16  #include <time.h>
17  #include <unistd.h>
18  #include <sys/stat.h>
19
20  #define TEST_FILE    "test_input.bin"
21  #define TEST_SIZE    (8 * 1024 * 1024) /* 8 MB test file */
22  #define MAX_BUFFER    8192
23
24  /* ----- Helpers ----- */
25
26  static double elapsed_ms(struct timespec start, struct timespec end)
27  {
28      return (end.tv_sec - start.tv_sec) * 1000.0 +
29             (end.tv_nsec - start.tv_nsec) / 100;
30  }
31
32  /* Writes the whole buffer, handling partial writes and interrupts. */
33  static int write_all(int fd, const char *buf, size_t len)
34  {
35      while (len > 0) {
36          ssize_t n = write(fd, buf, len);
37          if (n == -1) {
38              if (errno == EINTR) continue;
39              return -1;
40          }
41          buf += n;
42          len -= (size_t)n;
43      }
44      return 0;
45  }
46
47  /* ----- Copy using system calls ----- */
48
49  static long copy_syscalls(const char *src, const char *dst, size_t bufsize,
50                           long *syscall_count)
51  {
52      char buf[MAX_BUFFER];
53      long total = 0;
54      ssize_t n;
55
56      int in = open(src, O_RDONLY);
57      if (in == -1) { perror(src); return -1; }
58
59      /* lseek to the end gives the file size, then rewind to the start */
60      off_t size = lseek(in, 0, SEEK_END);
61      if (size == -1 || lseek(in, 0, SEEK_SET) == -1) {
62          perror("lseek"); close(in); return -1;
63      }
64
65      int out = open(dst, O_WRONLY | O_CREAT | O_TRUNC, 0644);
66      if (out == -1) { perror(dst); close(in); return -1; }
67
68      while ((n = read(in, buf, bufsize)) > 0) {
69          calls++;
70          if (write_all(out, buf, (size_t)n) == -1) {
71              perror("write"); close(in); close(out); return -1;
72          }
73      }
74
75      /* ----- Performance comparison ----- */
76
77  static void benchmark(void)
78  {
79      const size_t sizes[] = { 16, 4096 };
80      struct timespec t0, t1;
81
82      if (create_test_file() == -1) exit(EXIT_FAILURE);
83      printf("Test file: %s (%d MB)\n\n", TEST_FILE, TEST_SIZE / (1024 * 1024));
84      printf("%-28s %-8s %-12s %-12s\n",
85             "Method", "Buffer", "Time (ms)", "Syscalls", "Verified");
86      printf("-----\n");
87
88      for (int i = 0; i < 2; i++) {
89          long calls = 0;
90
91          clock_gettime(CLOCK_MONOTONIC, &t0);
92          copy_syscalls(TEST_FILE, "copy_syscall.bin", sizes[i], &calls);
93          clock_gettime(CLOCK_MONOTONIC, &t1);
94          printf("%-28s %-8zu %-12.2f %-12ld\n",
95                 "System calls (read/write)", sizes[i], elapsed_ms(t0, t1), calls,
96                 files_identical(TEST_FILE, "copy_syscall.bin") ? "yes" : "NO");
97
98          clock_gettime(CLOCK_MONOTONIC, &t0);
99          copy_stdio(TEST_FILE, "copy_stdio.bin", sizes[i]);
100         clock_gettime(CLOCK_MONOTONIC, &t1);
101         printf("%-28s %-8zu %-12.2f %-12s\n",
102                "Standard I/O (fread/fwrite)", sizes[i], elapsed_ms(t0, t1),
103                "buffered",
104                files_identical(TEST_FILE, "copy_stdio.bin") ? "yes" : "NO");
105     }
106
107     unlink(TEST_FILE);
108     unlink("copy_syscall.bin");
109     unlink("copy_stdio.bin");
110 }
111
112 }
113
114 if (n == -1) perror("read");
115
116 if (close(in) == -1 || close(out) == -1) { perror("close"); return -1; }
117 if (total != (long)size)
118     fprintf(stderr, "Warning: copied %ld of %ld bytes\n", total, (long)size);
119 if (*syscall_count * syscall_count = calls;
120     return total;
121 }
122
123 /* ----- Copy using the standard I/O Library ----- */
124
125 static long copy_stdio(const char *src, const char *dst, size_t bufsize)
126 {
127     char buf[MAX_BUFFER];
128     long total = 0;
129     size_t n;
130
131     FILE *in = fopen(src, "rb");
132     if (!in) { perror(src); return -1; }
133     FILE *out = fopen(dst, "wb");
134     if (!out) { perror(dst); fclose(in); return -1; }
135
136     while ((n = fread(buf, 1, bufsize, in)) > 0) {
137         if (fwrite(buf, 1, n, out) != n) {
138             perror("fwrite"); fclose(in); fclose(out); return -1;
139         }
140         total += (long)n;
141     }
142     if (ferror(in)) perror("fread");
143     fclose(in);
144     if (fclose(out) != 0) { perror("fclose"); return -1; }
145     return total;
146 }
147
148 /* ----- Verification: compare two files byte by byte ----- */
149
150 static int files_identical(const char *a, const char *b)
151 {
152     FILE *fa = fopen(a, "rb"), *fb = fopen(b, "rb");
153     int same = (fa && fb);
154     while (same) {
155         int ca = fgetc(fa), cb = fgetc(fb);
156         if (ca != cb) same = 0;
157         if (ca == EOF || cb == EOF) break;
158     }
159     if (fa) fclose(fa);
160     if (fb) fclose(fb);
161     return same;
162 }
163
164 static int create_test_file(void)
165 {
166     char block[4096];
167     int fd = open(TEST_FILE, O_WRONLY | O_CREAT | O_TRUNC, 0644);
168     if (fd == -1) { perror("create test file"); return -1; }
169     for (size_t i = 0; i < sizeof(block); i++)
170         block[i] = (char)('A' + i % 26);
171     for (int i = 0; i < TEST_SIZE / (int)sizeof(block); i++)
172         if (write_all(fd, block, sizeof(block)) == -1) {
173             return -1;
174         }
175     return 0;
176 }
177
178 int main(int argc, char *argv[])
179 {
180     if (argc == 3) {
181         long calls = 0;
182         long bytes = copy_syscalls(argv[1], argv[2], 4096, &calls);
183         if (bytes < 0) return EXIT_FAILURE;
184         printf("Copied %ld bytes from %s to %s using %ld read/write calls\n",
185                bytes, argv[1], argv[2], calls);
186         return EXIT_SUCCESS;
187     }
188     if (argc != 1) {
189         fprintf(stderr, "Usage: %s <source> <destination>\n", argv[0]);
190         fprintf(stderr, "        %s (run performance comparison)\n", argv[0]);
191         return EXIT_FAILURE;
192     }
193     benchmark();
194     return EXIT_SUCCESS;
195 }
```

Output

The exact addresses will be different on every execution because of Linux memory-layout randomization.

```
hrushika@Hrushikas-MacBook-Air pjsdj % ./w9q1
Test file: test_input.bin (8 MB)

Method                                Buffer   Time (ms)   Syscalls   Verified
-----
System calls (read/write)             16      1283.62     1048576    yes
Standard I/O (fread/fwrite)           16       47.55       buffered   yes
System calls (read/write)             4096     7.21        4096       yes
Standard I/O (fread/fwrite)           4096     16.01       buffered   yes
hrushika@Hrushikas-MacBook-Air pjsdj %
```

Observation

The program copied an 8 MB file four ways and verified that every copy is byte-for-byte identical to the original.

Method	Buffer	Behaviour
System calls	16 bytes	Every read() and write() enters the kernel. Over one million system calls were made, so this was by far the slowest method.
Standard I/O	16 bytes	fread()/fwrite() copy into a user-space buffer (typically 4 KB or larger) and call read()/write() only when it fills, so far fewer system calls are made.
System calls	4096 bytes	Each call moves a full block, so only about 4,096 system calls are needed. Performance matches standard I/O.
Standard I/O	4096 bytes	Similar to large-buffer system calls; the extra user-space copy adds very little cost.

- The cost of a copy depends mainly on the **number of system calls**, because each call requires a switch from user mode to kernel mode.
- lseek(fd, 0, SEEK_END) returns the file size, and lseek(fd, 0, SEEK_SET) moves the offset back to the start of the file.
- write() may write fewer bytes than requested, so the program loops until the whole buffer is written.
- Standard I/O is faster for small reads and writes because of its internal buffering; with a large buffer, raw system calls perform just as well and give direct control over the file descriptor.

Result

A file copy utility was successfully implemented using the low-level system calls **open()**, **read()**, **write()**, **lseek()** and **close()**. Its performance was compared with **fopen()**, **fread()** and **fwrite()**, showing that buffering, and therefore the number of system calls, determines file I/O performance.

QUESTION 2

Develop a program that redirects standard input and output using file descriptors and `dup2()`. Demonstrate how shells implement I/O redirection.

Aim:

To redirect standard input, standard output and standard error using file descriptors and the `dup()` and `dup2()` system calls, and to demonstrate how a shell implements the redirection operators `>`, `<`, `2>&1` using `fork()`, `dup2()` and `exec()`.

Program

```
*/
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/wait.h>

static void die(const char *msg)
{
    perror(msg);
    exit(EXIT_FAILURE);
}

static void show_file(const char *path)
{
    char line[256];
    FILE *fp = fopen(path, "r");
    if (!fp) die(path);
    while (fgets(line, sizeof(line), fp))
        printf("    | %s", line);
    fclose(fp);
}

int main(void)
{
    printf("Standard descriptors: stdin = %d, stdout = %d, stderr = %d\n\n",
        STDIN_FILENO, STDOUT_FILENO, STDERR_FILENO);

    /* ----- Part 1: output redirection (command > output.txt) ----- */
    printf("\nPart 1: Output redirection -> command > output.txt\n");
    fflush(stdout);

    int saved_stdout = dup(STDOUT_FILENO); /* keep terminal stdout */
    int fd = open("output.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (saved_stdout == -1 || fd == -1) die("open/dup");

    dup2(fd, STDOUT_FILENO); /* fd 1 now -> output.txt */
    close(fd);
    printf("This line was written by printf() into output.txt\n");
    printf("printf() still writes to fd 1, but fd 1 now points to the file\n");
    fflush(stdout);

    dup2(saved_stdout, STDOUT_FILENO); /* restore terminal */
    close(saved_stdout);
    printf(" stdout restored. Contents of output.txt:\n");
    show_file("output.txt");

    /* ----- Part 2: input redirection (command < input.txt) ----- */
    printf("\nPart 2: Input redirection -> command < input.txt\n");
    FILE *fp = fopen("input.txt", "r");
    if (!fp) die("input.txt");
    fprintf(fp, "Ravi\nAnita\nKiran\nDeepa\nArjun\n");
    fclose(fp);

    int saved_stdin = dup(STDIN_FILENO);
    fd = open("input.txt", O_RDONLY);
    if (saved_stdin == -1 || fd == -1) die("open/dup");
    dup2(fd, STDIN_FILENO); /* fd 0 now -> input.txt */
    close(fd);
```

```
dup2(fd, STDIN_FILENO); /* fd 0 now -> input.txt */
close(fd);

char name[64];
int count = 0;
while (fgets(name, sizeof(name), stdin)) { /* reads from the file */
    name[strlen(name)] = '\0';
    printf(" Read from stdin: %s\n", name);
    count++;
}

dup2(saved_stdin, STDIN_FILENO);
close(saved_stdin);
clearerr(stdin);
printf(" %d names read. stdin restored to the keyboard.\n", count);

/* ----- Part 3: error redirection (command 2>&1 > errors.txt) ----- */
printf("\nPart 3: Error redirection -> command > errors.txt 2>&1\n");
fflush(stdout);
pid_t pid = fork();
if (pid == -1) die("fork");
if (pid == 0) {
    fd = open("errors.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (fd == -1) die("errors.txt");
    dup2(fd, STDOUT_FILENO); /* > errors.txt */
    dup2(STDOUT_FILENO, STDERR_FILENO); /* 2>&1 */
    close(fd);
    execlp("ls", "ls", "input.txt", "missing_file.txt", (char *)NULL);
    die("execlp");
}

waitpid(pid, NULL, 0);
printf(" Both normal output and error message went to errors.txt\n");
show_file("errors.txt");

/* ----- Part 4: how a shell runs sort < input.txt > sorted.txt ----- */
printf("\nPart 4: Shell simulation -> sort < input.txt > sorted.txt\n");
fflush(stdout);
pid = fork();
if (pid == -1) die("fork");
if (pid == 0) {
    int in = open("input.txt", O_RDONLY);
    int out = open("sorted.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (in == -1 || out == -1) die("open");
    dup2(in, STDIN_FILENO); /* < input.txt */
    dup2(out, STDOUT_FILENO); /* > sorted.txt */
    close(in);
    close(out);
    execlp("sort", "sort", (char *)NULL); /* sort never knows about files */
    die("execlp");
}

int status;
waitpid(pid, &status, 0);
printf(" Child (PID %d) exited with status %d. Contents of sorted.txt:\n",
    pid, WEXITSTATUS(status));
show_file("sorted.txt");

unlink("output.txt");
unlink("input.txt");
unlink("errors.txt");
unlink("sorted.txt");
return EXIT_SUCCESS;
```

Output:

```
● hrushika@Hrushikas-MacBook-Air pjsdj % ./w9q1
Test file: test_input.bin (8 MB)

Method                Buffer   Time (ms)   Syscalls   Verified
-----
System calls (read/write) 16      1283.62    1048576    yes
Standard I/O (fread/fwrite) 16      47.55     buffered   yes
System calls (read/write) 4096     7.21      4096       yes
Standard I/O (fread/fwrite) 4096    16.01     buffered   yes
● hrushika@Hrushikas-MacBook-Air pjsdj % gcc w9q2.c -o w9q2
● hrushika@Hrushikas-MacBook-Air pjsdj % ./w9q2
Standard descriptors: stdin = 0, stdout = 1, stderr = 2

Part 1: Output redirection  ->  command > output.txt
      stdout restored. Contents of output.txt:
      | This line was written by printf() into output.txt
      | printf() still writes to fd 1, but fd 1 now points to the file

Part 2: Input redirection   ->  command < input.txt
      Read from stdin: Ravi
      Read from stdin: Anita
      Read from stdin: Kiran
      Read from stdin: Deepa
      Read from stdin: Arjun
      5 names read. stdin restored to the keyboard.

Part 3: Error redirection   ->  command > errors.txt 2>&1
      Both normal output and error message went to errors.txt:
      | ls: missing_file.txt: No such file or directory
      | input.txt

Part 4: Shell simulation     ->  sort < input.txt > sorted.txt
      Child (PID 99387) exited with status 0. Contents of sorted.txt:
      | Anita
      | Arjun
      | Deepa
      | Kiran
      | Ravi
❖ hrushika@Hrushikas-MacBook-Air pjsdj %
```

Observation

Every process starts with three open file descriptors: 0 (stdin), 1 (stdout) and 2 (stderr). `dup2(oldfd, newfd)` makes `newfd` refer to the same open file as `oldfd`, so any code that uses `newfd` is silently redirected.

Shell syntax	Equivalent system calls	Effect
cmd > file	<code>fd = open(file, O_WRONLY O_CREAT O_TRUNC); dup2(fd, 1)</code>	Standard output is written to the file
cmd < file	<code>fd = open(file, O_RDONLY); dup2(fd, 0)</code>	Standard input is read from the file
cmd 2>&1	<code>dup2(1, 2)</code>	Errors go wherever standard output goes
sort < in > out	<code>fork();</code> in the child: <code>dup2</code> for <code>fd 0</code> and <code>fd 1</code> , then <code>execlp("sort")</code>	The shell sets up the descriptors; <code>sort</code> itself is unaware of the files

- `printf()` always writes to descriptor 1. After `dup2(fd, 1)` its output went to `output.txt` without any change to the `printf()` calls.
- Saving the original descriptor with `dup()` allowed the program to restore the terminal afterwards.
- Open file descriptors are inherited across `fork()` and preserved across `exec()`. This is exactly how a shell redirects the I/O of commands such as `sort` and `ls`.
- The unused descriptor is closed after `dup2()` so that no descriptors are leaked.

Result

Standard input, output and error were successfully redirected using **`dup()`** and **`dup2()`**. The program demonstrated how a shell implements output redirection, input redirection, error redirection and combined redirection using **`fork()`**, **`dup2()`** and **`exec()`**.